

MPI USING MATRIX MULTIPLICATION

Distributed-Memory Matrix Multiplication Using Open MPI Across Four Ubuntu Virtual Machines

Experiment	MPI Distributed Matrix Multiplication
Architecture	1 Master VM + 3 Worker VMs
Matrix Size	4000 x 4000
MPI Processes	4
Execution Time	92.979510 seconds
Verification	C[0][0] = 4000.00

1. Aim

To implement distributed-memory matrix multiplication using the Message Passing Interface (MPI) across four Ubuntu virtual machines. The experiment configures one Master node and three Worker nodes, establishes network and SSH communication, launches MPI processes on all nodes, demonstrates point-to-point message passing, distributes matrix data using MPI collective operations, performs parallel matrix multiplication, gathers the final result, measures execution time, and verifies correctness.

2. Objectives

1. Create one Master VM and three Worker VMs.
2. Configure unique hostnames and IP addresses.
3. Verify network connectivity between all cluster nodes.
4. Configure SSH and passwordless remote access.
5. Install Open MPI on every node.
6. Create an MPI hostfile.
7. Verify MPI process execution on four VMs.
8. Demonstrate `MPI_Send()` and `MPI_Recv()`.
9. Distribute Matrix A using `MPI_Scatter()`.
10. Broadcast Matrix B using `MPI_Bcast()`.
11. Compute separate rows of Matrix C on each MPI rank.
12. Collect local results using `MPI_Gather()`.
13. Measure execution time using `MPI_Wtime()`.
14. Verify the final matrix result.

3. Introduction to MPI

MPI stands for **Message Passing Interface**. It is a standardized programming model for communication among multiple processes. These processes may execute on the same computer or on different computers connected through a network.

MPI follows a distributed-process model. Every MPI process has its own address space. A variable created by one process is not automatically accessible to another process. Data must be communicated explicitly using point-to-point operations such as `MPI_Send` and `MPI_Recv` or collective operations such as `MPI_Scatter`, `MPI_Bcast`, and `MPI_Gather`.

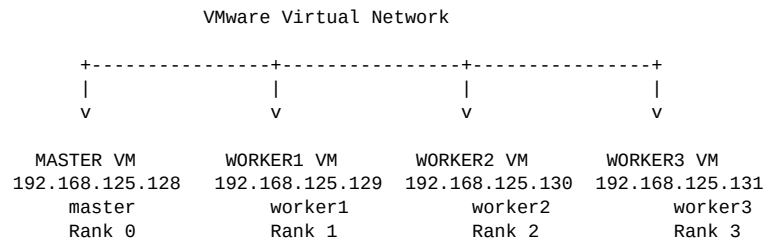
```
OpenMP:
One process -> multiple threads -> shared memory

MPI:
Rank 0 memory   Rank 1 memory   Rank 2 memory   Rank 3 memory
    |             |             |             |
    +----- explicit MPI communication -----+
```

4. MPI Cluster Architecture

The cluster consists of four Ubuntu virtual machines connected through the VMware virtual network. The Master launches the MPI application. Three additional VMs participate as worker nodes.

Node	Hostname	IP Address	MPI Role
Master	master	192.168.125.128	Rank 0 / root process
Worker1	worker1	192.168.125.129	Rank 1
Worker2	worker2	192.168.125.130	Rank 2
Worker3	worker3	192.168.125.131	Rank 3



The Master VM was configured with one virtual processor, eight vCPUs, and 8 GB RAM. The Worker VMs were created as separate virtual machines. All nodes are connected to the same virtual network and communicate using their assigned IP addresses.

5. Software Requirements

Software	Purpose
Ubuntu 24.04 LTS	Operating system
OpenSSH Server	Remote access and MPI remote process launch
Open MPI 4.1.6	MPI implementation
mpicc	Compiles MPI C programs
mpirun	Launches MPI processes
GCC	Underlying C compiler
scp	Copies executables between nodes
VMware	Virtual machine platform

6. Step 1 - Configure Unique Hostnames

Every node in the MPI cluster must have a unique hostname. Run the following command on each VM to verify its hostname:

```
hostname
```

The required hostname configuration is:

```
Master : master
Worker1 : worker1
Worker2 : worker2
Worker3 : worker3
```

To assign a hostname, use the corresponding command on each node:

```
sudo hostnamectl set-hostname master
sudo hostnamectl set-hostname worker1
sudo hostnamectl set-hostname worker2
sudo hostnamectl set-hostname worker3
```

Execute only the hostname command appropriate for the current VM. Reboot the VM if required and verify again using hostname.

7. Step 2 - Identify IP Addresses

Run the following command on every VM:

```
hostname -I
```

The cluster uses the following IP mapping:

Node	IP Address
Master	192.168.125.128
Worker1	192.168.125.129
Worker2	192.168.125.130
Worker3	192.168.125.131

8. Step 3 - Verify Network Connectivity

From the Master VM, test communication with every Worker VM using ping:

```
ping -c 4 192.168.125.129
ping -c 4 192.168.125.130
ping -c 4 192.168.125.131
```

Expected result for every worker:

```
4 packets transmitted, 4 received, 0% packet loss
```

A 0% packet-loss result confirms that the Master can reach the Worker VMs through the virtual network. Network connectivity is required before configuring distributed MPI execution.

9. Step 4 - Install and Enable SSH

Open MPI requires remote access to start MPI processes on worker nodes. Install the OpenSSH server on each Worker VM:

```
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable --now ssh
```

Verify the SSH service:

```
sudo systemctl status ssh
```

Expected service state:

```
Active: active (running)
```

10. Step 5 - Test SSH Communication

From the Master VM, connect to each worker:

```
ssh worker1@worker1
ssh worker1@worker2
ssh worker1@worker3
```

After connecting, execute `hostname` to confirm the remote node. Use `exit` to return to the Master.

11. Step 6 - Configure Passwordless SSH

MPI process launching should occur without interactive password entry. Generate an RSA SSH key on the Master:

```
ssh-keygen -t rsa
```

Accept the default key location. Copy the public key to each worker:

```
ssh-copy-id worker1@worker1
ssh-copy-id worker1@worker2
ssh-copy-id worker1@worker3
```

Test remote access again. The Master should be able to connect to each worker without entering a password.

12. Step 7 - Configure SSH Host Aliases

The Master uses the Linux username `master`, while the Worker VMs use the username `worker1`. Configure SSH aliases on the Master so that each worker hostname automatically uses the correct remote username.

```
nano ~/.ssh/config
```

Add:

```
Host worker1
  HostName worker1
  User worker1

Host worker2
  HostName worker2
  User worker1

Host worker3
  HostName worker3
  User worker1
```

Set secure permissions:

```
chmod 600 ~/.ssh/config
```

Verify:

```
ssh worker1 hostname  
ssh worker2 hostname  
ssh worker3 hostname
```

Expected output:

```
worker1  
worker2  
worker3
```

13. Step 8 - Install Open MPI on All Nodes

The same MPI implementation must be available on the Master and all Worker VMs. Execute the following commands on every node:

```
sudo apt update
sudo apt install openmpi-bin libopenmpi-dev -y
```

Verify the MPI installation:

```
mpirun --version
mpicc --version
```

The MPI environment used for the experiment is Open MPI 4.1.6. mpirun is the process launcher. mpicc is an MPI-aware compiler wrapper that invokes the underlying C compiler with the required MPI header and library configuration.

14. Step 9 - Create the MPI Hostfile

On the Master VM, create a hostfile:

```
nano hosts
```

Enter the following configuration:

```
master slots=1
worker1 slots=1
worker2 slots=1
worker3 slots=1
```

Save the file and verify it:

```
cat hosts
```

The hostfile defines four MPI process slots, one on each node. For this experiment, one MPI process is assigned to each VM.

15. Step 10 - Verify Distributed MPI Process Launching

Execute the hostname command using four MPI processes:

```
mpirun -np 4 --hostfile hosts hostname
```

Expected node output:

```
master
worker1
worker2
worker3
```

The order of the hostnames may vary because MPI processes execute concurrently. The presence of all four hostnames confirms that MPI can launch a process on every configured VM.

16. Step 11 - MPI Send and Receive Program

Before implementing matrix multiplication, a point-to-point communication program is used to demonstrate explicit message passing. Rank 0 creates an integer value $A = 10$ and sends it to Rank 1. Rank 1 receives the value and prints it.

```
nano mpi_send_recv.c
```

Program:

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int A;
    char hostname[256];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    gethostname(hostname, sizeof(hostname));

    printf("Rank %d is running on %s\n", rank, hostname);

    if (size < 2)
    {
        if (rank == 0)
            printf("This program requires at least 2 MPI processes.\n");

        MPI_Finalize();
        return 0;
    }

    if (rank == 0)
    {
        A = 10;

        printf("Rank 0 on %s: Sending A = %d to Rank 1\n",
            hostname, A);

        MPI_Send(&A, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        MPI_Recv(&A, 1, MPI_INT, 0, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Rank 1 on %s: Received A = %d from Rank 0\n",
            hostname, A);
    }

    MPI_Finalize();
    return 0;
}
```

17. Compile the MPI Send/Receive Program

Compile the source file on the Master:

```
mpicc mpi_send_recv.c -o mpi_send_recv
```

Verify the executable:

```
ls -l mpi_send_recv
```

Expected result: an executable file named `mpi_send_recv` is displayed.

18. Copy the Executable to Worker Nodes

The four VMs use separate filesystems. Therefore, the executable is copied to each Worker VM using `scp`:

```
scp mpi_send_recv worker1:~/
scp mpi_send_recv worker2:~/
scp mpi_send_recv worker3:~/
```

Verify the executable on each worker:

```
ssh worker1 "ls -l ~/mpi_send_recv"  
ssh worker2 "ls -l ~/mpi_send_recv"  
ssh worker3 "ls -l ~/mpi_send_recv"
```

19. Run the MPI Send/Receive Program

```
env -u DISPLAY mpirun -np 4 --hostfile hosts sh -c '$HOME/mpi_send_recv'
```

Expected output:

```
Rank 0 is running on master
Rank 0 on master: Sending A = 10 to Rank 1
Rank 1 is running on worker1
Rank 2 is running on worker2
Rank 3 is running on worker3
Rank 1 on worker1: Received A = 10 from Rank 0
```

20. Explanation of MPI_Send and MPI_Recv

20.1 MPI Initialization

```
MPI_Init(&argc, &argv);
```

Initializes the MPI execution environment.

20.2 Rank Identification

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

Assigns a unique rank to each process. The ranks in the experiment are 0, 1, 2, and 3.

20.3 Process Count

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Returns the total number of MPI processes. The value is 4.

20.4 Sending the Integer

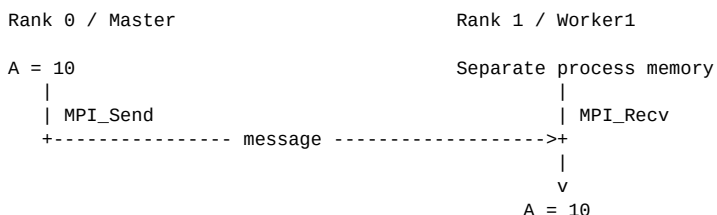
```
MPI_Send(&A, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

Parameter	Meaning
&A	Address of send buffer
1	Number of elements
MPI_INT	Integer data type
1	Destination Rank 1
0	Message tag
MPI_COMM_WORLD	MPI communicator

20.5 Receiving the Integer

```
MPI_Recv(&A, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Rank 1 receives one integer from Rank 0 with message tag 0 and stores the received value in its local variable A.



This demonstrates distributed memory. Rank 1 does not directly access Rank 0's variable. The value is explicitly transferred using MPI communication.

21. Matrix Multiplication Problem

The main MPI application performs multiplication of two 4000 x 4000 matrices. The mathematical operation is $C = A \times B$. Every element of A and B is initialized to 1.0.

For every output element $C[i][j]$, 4000 multiplication results are added. Since each multiplication is 1×1 , the expected output value is 4000.

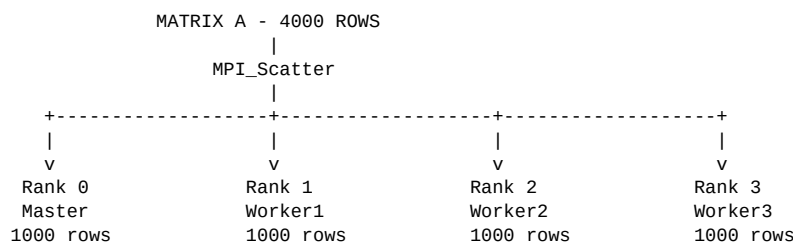
```
C[i][j] = A[i][0] * B[0][j]
        + A[i][1] * B[1][j]
        + ...
        + A[i][3999] * B[3999][j]

C[i][j] = 1 + 1 + ... + 1
        = 4000
```

22. Distribution of Matrix A

The 4000 rows of Matrix A are divided equally among four MPI processes. The number of rows per process is calculated as:

```
rows_per_process = N / size
                  = 4000 / 4
                  = 1000 rows
```



Rank	Node	Rows of A
0	master	0-999
1	worker1	1000-1999
2	worker2	2000-2999
3	worker3	3000-3999

23. Distribution of Matrix B

Each rank computes complete output columns for its assigned rows. Therefore, every rank requires the entire Matrix B. MPI_Bcast broadcasts Matrix B from Rank 0 to all processes.

24. Collection of Matrix C

Each MPI rank computes a local_C buffer containing 1000 rows. MPI_Gather collects the four local result buffers and reconstructs the complete Matrix C on Rank 0.

```
Rank 0 local_C -----+
Rank 1 local_C -----|
Rank 2 local_C -----+----- MPI_Gather ----> Complete Matrix C on Rank 0
Rank 3 local_C -----+
```

25. Complete MPI Matrix Multiplication Program

Create the source file on the Master:

```
nano matrix_mpi.c
```

Enter the following program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <unistd.h>

#define N 4000

int main(int argc, char *argv[])
{
    int rank, size;
    int i, j, k;
    int rows_per_process;
    char hostname[256];

    double *A = NULL;
    double *B = NULL;
    double *C = NULL;
    double *local_A;
    double *local_C;

    double start, end;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    gethostname(hostname, sizeof(hostname));

    if (N % size != 0)
    {
        if (rank == 0)
            printf("Matrix size must be divisible by number of processes.\n");

        MPI_Finalize();
        return 0;
    }

    rows_per_process = N / size;

    local_A = (double *)malloc(
        rows_per_process * N * sizeof(double));

    local_C = (double *)malloc(
        rows_per_process * N * sizeof(double));

    B = (double *)malloc(
        N * N * sizeof(double));

    if (rank == 0)
    {
        A = (double *)malloc(N * N * sizeof(double));
        C = (double *)malloc(N * N * sizeof(double));

        printf("Initializing %d x %d matrices...\n", N, N);

        for (i = 0; i < N; i++)
        {
            for (j = 0; j < N; j++)
            {
                A[i * N + j] = 1.0;
                B[i * N + j] = 1.0;
                C[i * N + j] = 0.0;
            }
        }
    }

    MPI_Barrier(MPI_COMM_WORLD);

    start = MPI_Wtime();

    MPI_Scatter(
        A,
        rows_per_process * N,
        MPI_DOUBLE,
        local_A,
        rows_per_process * N,
        MPI_DOUBLE,
        0,
        MPI_COMM_WORLD);

    MPI_Bcast(
        B,
        N * N,
        MPI_DOUBLE,
```

```

    0,
    MPI_COMM_WORLD);

printf("Rank %d on %s computing %d rows\n",
      rank, hostname, rows_per_process);

for (i = 0; i < rows_per_process; i++)
{
    for (j = 0; j < N; j++)
    {
        local_C[i * N + j] = 0.0;

        for (k = 0; k < N; k++)
        {
            local_C[i * N + j] +=
                local_A[i * N + k] *
                B[k * N + j];
        }
    }
}

MPI_Gather(
    local_C,
    rows_per_process * N,
    MPI_DOUBLE,
    C,
    rows_per_process * N,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD);

MPI_Barrier(MPI_COMM_WORLD);

end = MPI_Wtime();

if (rank == 0)
{
    printf("\nMPI Matrix Multiplication Completed\n");
    printf("Matrix Size = %d x %d\n", N, N);
    printf("Number of MPI Processes = %d\n", size);
    printf("Execution Time = %f seconds\n", end - start);
    printf("Verification C[0][0] = %.2f\n", C[0]);

    free(A);
    free(C);
}

free(B);
free(local_A);
free(local_C);

MPI_Finalize();

return 0;
}

```

26. Detailed Program Explanation

26.1 Matrix Dimension

```
#define N 4000
```

Defines both matrices as 4000 x 4000.

26.2 Dynamic Memory Allocation

Matrices are allocated dynamically using malloc. A 4000 x 4000 matrix contains 16,000,000 double values. With an 8-byte double, one full matrix occupies approximately 122 MiB.

Buffer	Purpose
A	Complete Matrix A on Rank 0
B	Complete Matrix B on every rank
C	Complete Matrix C on Rank 0
local_A	1000-row A chunk on each rank
local_C	1000-row result chunk on each rank

26.3 MPI_Init

```
MPI_Init(&argc, &argv);
```

Starts the MPI environment.

26.4 MPI_Comm_rank and MPI_Comm_size

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Every process identifies its rank and obtains the total process count.

26.5 Equal Row Division

```
rows_per_process = N / size;
```

The 4000 rows are equally divided among four processes, giving 1000 rows per rank.

26.6 Matrix Initialization

Rank 0 allocates full Matrix A and Matrix C. Matrix A and Matrix B are initialized to 1.0. Matrix C is initialized to 0.0.

26.7 Synchronization Before Timing

```
MPI_Barrier(MPI_COMM_WORLD);  
start = MPI_Wtime();
```

MPI_Barrier synchronizes all ranks before the distributed phase. MPI_Wtime records the wall-clock start time.

26.8 MPI_Scatter

```
MPI_Scatter(  
    A,  
    rows_per_process * N,  
    MPI_DOUBLE,  
    local_A,  
    rows_per_process * N,  
    MPI_DOUBLE,  
    0,  
    MPI_COMM_WORLD);
```

MPI_Scatter divides Matrix A into four equal contiguous row chunks. Each rank receives 1000 x 4000 double values in local_A.

26.9 MPI_Bcast

```
MPI_Bcast(B, N * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

MPI_Bcast distributes the complete Matrix B from Rank 0 to every MPI process.

26.10 Local Matrix Multiplication

```
for (i = 0; i < rows_per_process; i++)
{
    for (j = 0; j < N; j++)
    {
        local_C[i * N + j] = 0.0;

        for (k = 0; k < N; k++)
        {
            local_C[i * N + j] +=
                local_A[i * N + k] *
                B[k * N + j];
        }
    }
}
```

Each rank executes the standard matrix multiplication algorithm for only its 1000 local rows. The four ranks perform their local computations concurrently.

```
Rank 0 on master -> computes 1000 rows
Rank 1 on worker1 -> computes 1000 rows
Rank 2 on worker2 -> computes 1000 rows
Rank 3 on worker3 -> computes 1000 rows
```

26.11 MPI_Gather

```
MPI_Gather(
    local_C,
    rows_per_process * N,
    MPI_DOUBLE,
    C,
    rows_per_process * N,
    MPI_DOUBLE,
    0,
    MPI_COMM_WORLD);
```

MPI_Gather collects the local_C buffers from all ranks and places them in rank order inside the complete Matrix C on Rank 0.

26.12 End Time and Verification

```
MPI_Barrier(MPI_COMM_WORLD);
end = MPI_Wtime();

printf("Execution Time = %f seconds\n", end - start);
printf("Verification C[0][0] = %.2f\n", C[0]);
```

The final barrier synchronizes all processes. The elapsed time includes MPI communication, local matrix computation, result gathering, and synchronization. Rank 0 prints the execution time and verifies C[0][0].

26.13 MPI_Finalize

```
MPI_Finalize();
```

Terminates the MPI environment cleanly on every process.

27. Compile the Matrix MPI Program

On the Master VM:

```
mpicc matrix_mpi.c -o matrix_mpi
ls -l matrix_mpi
```

The executable matrix_mpi is created.

28. Copy the Matrix Executable to Workers

```
scp matrix_mpi worker1:~/
scp matrix_mpi worker2:~/
scp matrix_mpi worker3:~/
```

Verify the executable on each worker:

```
ssh worker1 "ls -l ~/matrix_mpi"
ssh worker2 "ls -l ~/matrix_mpi"
ssh worker3 "ls -l ~/matrix_mpi"
```

29. Execute Distributed Matrix Multiplication

Run the MPI application from the Master:

```
env -u DISPLAY mpirun -np 4 --hostfile hosts sh -c '$HOME/matrix_mpi'
```

Command Part	Purpose
env -u DISPLAY	Runs with DISPLAY removed from the command environment
mpirun	Launches the MPI job
-np 4	Starts four MPI processes
--hostfile hosts	Uses the configured cluster nodes
sh -c	Executes a shell command on the launched node
\$HOME/matrix_mpi	Runs the executable from each node's local home directory

30. Expected Output

```
Initializing 4000 x 4000 matrices...
Rank 0 on master computing 1000 rows
Rank 1 on worker1 computing 1000 rows
Rank 2 on worker2 computing 1000 rows
Rank 3 on worker3 computing 1000 rows

MPI Matrix Multiplication Completed
Matrix Size = 4000 x 4000
Number of MPI Processes = 4
Execution Time = [measured time] seconds
Verification C[0][0] = 4000.00
```

31. Final Experimental Output

```
Initializing 4000 x 4000 matrices...
Rank 0 on master computing 1000 rows
Rank 1 on worker1 computing 1000 rows
Rank 2 on worker2 computing 1000 rows
Rank 3 on worker3 computing 1000 rows
```

```
MPI Matrix Multiplication Completed
Matrix Size = 4000 x 4000
Number of MPI Processes = 4
Execution Time = 92.979510 seconds
Verification C[0][0] = 4000.00
```

32. Interpretation of the Final Result

32.1 Four MPI Processes

The output identifies Rank 0 on master, Rank 1 on worker1, Rank 2 on worker2, and Rank 3 on worker3. This confirms that all four VMs participated in the MPI application.

32.2 Equal Work Distribution

Every rank reports that it is computing 1000 rows. The 4000 rows of Matrix A were therefore divided equally across the four MPI processes.

32.3 Execution Time

The distributed MPI phase completed in **92.979510 seconds**. The timed section includes data distribution, Matrix B broadcast, local computation, result collection, and synchronization.

32.4 Result Verification

The program reported **C[0][0] = 4000.00**. Since Matrix A and Matrix B contain only 1.0 values, the expected dot product is 4000.00. The actual result matches the expected value.

33. Complete MPI Execution Flow

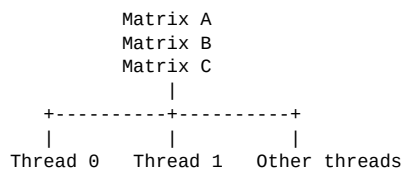
1. Master executes mpirun.
2. Open MPI launches four processes.
3. Rank 0 runs on master.
4. Rank 1 runs on worker1.
5. Rank 2 runs on worker2.
6. Rank 3 runs on worker3.
7. Every process calls MPI_Init.
8. Each process obtains its rank and process count.
9. Rank 0 initializes full Matrix A and Matrix B.
10. All ranks synchronize.
11. MPI_Wtime starts the timer.
12. MPI_Scatter distributes 1000 rows of A to each rank.
13. MPI_Bcast distributes complete B to every rank.
14. Four ranks compute separate result rows concurrently.
15. Every rank stores its result in local_C.
16. MPI_Gather collects all local_C buffers on Rank 0.
17. The complete Matrix C is available on the Master.
18. All ranks synchronize.
19. MPI_Wtime records the end time.
20. Rank 0 prints execution time and verifies C[0][0].
21. Dynamically allocated memory is released.
22. Every process calls MPI_Finalize.

34. Distributed Memory Architecture

The experiment demonstrates the major difference between shared-memory and distributed-memory parallel programming.

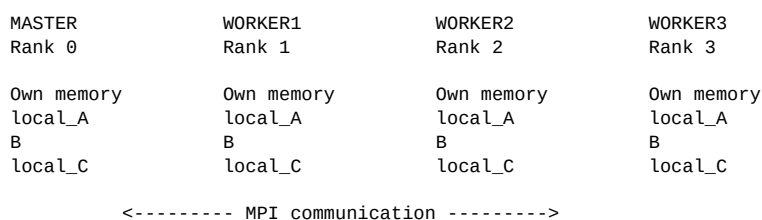
34.1 Shared-Memory Model

One process address space



Threads can access shared matrix memory.

34.2 MPI Distributed-Memory Model



Each MPI rank is an independent process. A rank cannot directly access another rank's local variables or pointers. MPI operations explicitly transfer data between process memories.

35. MPI Functions Used

MPI Function	Purpose
MPI_Init	Initializes MPI
MPI_Comm_rank	Obtains the rank number
MPI_Comm_size	Obtains the number of processes
MPI_Send	Sends a message to another rank
MPI_Recv	Receives a message from another rank
MPI_Barrier	Synchronizes all ranks
MPI_Wtime	Measures wall-clock time
MPI_Scatter	Distributes equal data chunks
MPI_Bcast	Broadcasts data to all ranks
MPI_Gather	Collects result chunks on the root
MPI_Finalize	Terminates MPI

36. Performance Observation

The recorded execution times for the matrix multiplication learning experiments were:

Implementation	Configuration	Execution Time
Sequential	Single process	Approximately 244 seconds
OpenMP	8 threads	30.830434 seconds
MPI	4 ranks across 4 VMs	92.979510 seconds

The MPI implementation reduced execution time compared with the recorded sequential execution. The OpenMP implementation was faster than the MPI implementation in this particular virtualized environment.

36.1 MPI Speedup

$\text{Speedup} = \text{Sequential Time} / \text{MPI Time}$

$\text{MPI Speedup} = 244 / 92.979510$
= approximately 2.62 x

36.2 MPI Parallel Efficiency

$\text{Efficiency} = \text{Speedup} / \text{Number of MPI Processes}$

$\text{MPI Efficiency} = 2.62 / 4$
= approximately 65.6%

37. Why MPI Performance Differs from OpenMP

OpenMP and MPI use different parallel execution models. OpenMP uses threads inside one shared-memory process. MPI uses separate processes and explicit communication.

In this experiment, the OpenMP program used eight threads, whereas the MPI program used four single-threaded MPI ranks. The MPI program also communicates large matrix buffers using MPI_Scatter, MPI_Bcast, and MPI_Gather.

The four MPI VMs are hosted on the same physical computer. Therefore, the virtual machines share the physical processor and memory subsystem. The cluster demonstrates distributed-memory software architecture, but it does not provide four independent physical computers. This affects performance and explains why the MPI result should be interpreted in the context of the VMware test environment.

For a rigorous performance evaluation, every configuration should be executed multiple times under the same conditions. The mean execution time and standard deviation should be reported.

38. Complete Command Sequence

```
# Check hostname and IP
hostname
hostname -I

# Test network from Master
ping -c 4 192.168.125.129
ping -c 4 192.168.125.130
ping -c 4 192.168.125.131

# Install SSH on workers
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable --now ssh
sudo systemctl status ssh

# Generate SSH key on Master
ssh-keygen -t rsa

# Copy public key
ssh-copy-id worker1@worker1
ssh-copy-id worker1@worker2
ssh-copy-id worker1@worker3

# Configure SSH aliases on Master
nano ~/.ssh/config
chmod 600 ~/.ssh/config

# Verify passwordless remote access
ssh worker1 hostname
ssh worker2 hostname
ssh worker3 hostname

# Install Open MPI on all nodes
sudo apt update
sudo apt install openmpi-bin libopenmpi-dev -y

# Verify MPI
mpirun --version
mpicc --version

# Create hostfile on Master
nano hosts
cat hosts

# Verify distributed process launch
mpirun -np 4 --hostfile hosts hostname

# Create and compile send/receive program
nano mpi_send_recv.c
mpicc mpi_send_recv.c -o mpi_send_recv

# Copy send/receive executable
scp mpi_send_recv worker1:~/
scp mpi_send_recv worker2:~/
scp mpi_send_recv worker3:~/

# Run MPI send/receive
env -u DISPLAY mpirun -np 4 --hostfile hosts sh -c '$HOME/mpi_send_recv'

# Create and compile matrix program
nano matrix_mpi.c
mpicc matrix_mpi.c -o matrix_mpi
ls -l matrix_mpi

# Copy matrix executable
scp matrix_mpi worker1:~/
scp matrix_mpi worker2:~/
scp matrix_mpi worker3:~/

# Verify matrix executable
ssh worker1 "ls -l ~/matrix_mpi"
ssh worker2 "ls -l ~/matrix_mpi"
ssh worker3 "ls -l ~/matrix_mpi"

# Execute distributed matrix multiplication
env -u DISPLAY mpirun -np 4 --hostfile hosts sh -c '$HOME/matrix_mpi'
```

39. Result

A four-node MPI cluster consisting of one Master VM and three Worker VMs was successfully configured. All nodes were assigned unique hostnames and IP addresses. Network communication was verified with 0% packet loss. SSH and passwordless remote access were configured, and Open MPI 4.1.6 was installed across the cluster.

MPI process launching was verified across master, worker1, worker2, and worker3. Point-to-point message passing was demonstrated by sending the integer value 10 from Rank 0 to Rank 1 using MPI_Send and MPI_Recv.

A 4000 x 4000 matrix multiplication was then distributed across four MPI processes. MPI_Scatter distributed 1000 rows of Matrix A to every rank. MPI_Bcast distributed the complete Matrix B. Each MPI process computed 1000 rows of Matrix C. MPI_Gather collected the local result chunks and reconstructed the complete Matrix C on Rank 0.

The final MPI execution time was **92.979510 seconds**. The computed value **C[0][0] = 4000.00** matched the expected result.

40. Conclusion

The experiment successfully implemented distributed-memory matrix multiplication using MPI across four Ubuntu virtual machines. The complete cluster was configured using one Master and three Worker nodes connected through a virtual network.

The point-to-point communication experiment demonstrated the basic MPI principle that independent processes have separate memory spaces and exchange data through explicit messages. Rank 0 sent the value 10 to Rank 1 using MPI_Send, and Rank 1 received the value using MPI_Recv.

The matrix multiplication application demonstrated collective MPI communication. MPI_Scatter divided Matrix A into equal row blocks, MPI_Bcast distributed Matrix B to all ranks, each process independently computed its assigned rows, and MPI_Gather collected the distributed results on the Master.

The final output confirmed participation of all four VMs. Each rank computed 1000 rows, and the program completed the 4000 x 4000 matrix multiplication in 92.979510 seconds. The verification value C[0][0] = 4000.00 matched the expected mathematical result.

Therefore, distributed matrix multiplication using MPI was successfully implemented and verified across one Master VM and three Worker VMs.

41. Final Experimental Summary

Parameter	Value
Experiment	MPI USING MATRIX MULTIPLICATION
Cluster Size	4 Virtual Machines
Master	master - 192.168.125.128
Worker1	worker1 - 192.168.125.129
Worker2	worker2 - 192.168.125.130
Worker3	worker3 - 192.168.125.131
MPI Version	Open MPI 4.1.6
Matrix Size	4000 x 4000

Parameter	Value
MPI Processes	4
Rows per Process	1000
Execution Time	92.979510 seconds
Expected C[0][0]	4000.00
Actual C[0][0]	4000.00
Status	SUCCESSFUL

END OF MANUAL